

INTRODUCTION TO MICROPROCESSOR

Nowadays, microprocessor can be seen in almost all types of electronics devices like mobile phones, printers, washing machines etc. Microprocessors are also used in advanced applications like radars, satellites and flights. Due to the rapid advancements in electronic industry and large-scale integration of devices results in a significant cost reduction and increase application of microprocessors and their derivatives.

A *microprocessor* is a programmable electronics chip that has computing and decision making capabilities similar to central processing unit of a computer. Any microprocessor-based systems having limited number of resources are called *microcomputers*.

A microprocessor

A microprocessor is a semiconductor component that incorporates the functions of a central processing unit (CPU) on a single Integrated Circuit (IC). i.e., the central processing unit (CPU) built on a single IC is called microprocessor.

Microprocessor is a multipurpose, programmable, clock-driven, register based electronic device that reads binary instructions from a storage device called memory, accepts binary data as input and processes data according to those instructions, and provides as output.

Microprocessor-based system

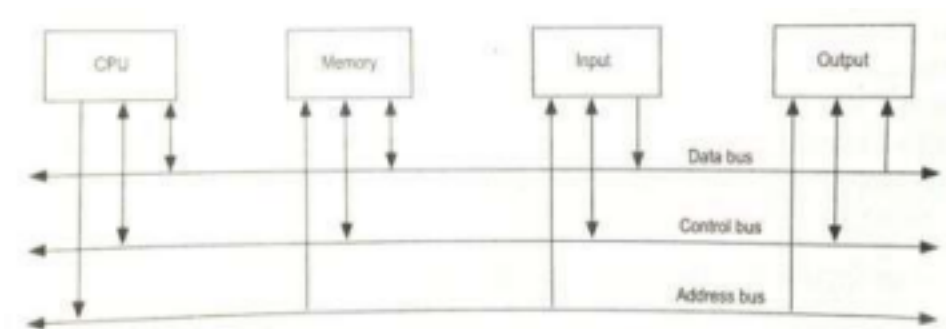


Fig.1 Microprocessor-based system

- **Bit:** A bit is a single binary digit.
- **Word:** A word refers to the basic data size or bit size that can be processed by the arithmetic and logic unit of the processor. A 16-bit binary number is called a word in a 16-bit processor.
- **Bus:** A bus is a group of wires/lines that carry similar information.
- **System Bus:** The system bus is a group of wires/lines used for communication between the microprocessor and peripherals.
- **Memory Word:** The number of bits that can be stored in a register or memory element is called a memory word.
- **Address Bus:** It carries the address, which is a unique binary pattern used to identify a memory location or an I/O port. For example, an eight-bit address bus has eight lines and thus it can address $2^8 = 256$ different locations. The locations in hexadecimal format can be written as 00H – FFH.
- **Data Bus:** The data bus is used to transfer data between memory and processor or between I/O device and processor. For example, an 8-bit processor will generally have an 8-bit data bus and a 16-bit processor will have 16-bit data bus.
- **Control Bus:** The control bus carry control signals, which consists of signals for selection of memory or I/O device from

the given address, direction of data transfer and synchronization of data transfer in case of slow devices.

To do any task using a microprocessor, it is to be programmed by the user. So, the programmer must have idea about its internal resources, features and supported instructions.

Each microprocessor has a set of instructions, a list which is provided by the microprocessor manufacturer. The instruction set of a microprocessor is provided in two forms: *binary machine code and mnemonics*.

Microprocessor communicates and operates in binary numbers 0 and 1. The set of instructions in the form of binary patterns is called a *machine language* and it is difficult for us to understand. Therefore, the binary patterns are given abbreviated names, called mnemonics, which forms the *assembly language*. The conversion of assembly-level language into binary machine-level language is done by using an application called *assembler*.

Technology Used:

The semiconductor manufacturing technologies used for chips are:

- Transistor-Transistor Logic (TTL)
- Emitter Coupled Logic (ECL)
- Complementary Metal-Oxide Semiconductor (CMOS)

Classification of Microprocessors:

Based on their specification, application and architecture microprocessors are classified. ***Based on size of data bus:***

- 4-bit microprocessor
- 8-bit microprocessor
- 16-bit microprocessor
- 32-bit microprocessor

Based on application:

- General-purpose microprocessor- used in general computer system and can be used by programmer for any application.

Examples, 8085 to Intel Pentium.

- Microcontroller- microprocessor with built-in memory and ports and can be programmed for any generic control application. Example, 8051.
- Special-purpose processors- designed to handle special functions required for an application. Examples, digital signal processors and application-specific integrated circuit (ASIC) chips.

Based on architecture:

- Reduced Instruction Set Computer (RISC) processors
- Complex Instruction Set Computer (CISC)processors

FUNCTIONAL BLOCK DIAGRAM OF MICROPROCESSOR

A typical microprocessor consists of register array, arithmetic and logic unit (ALU) in association with control unit to

process the instruction execution. Almost all the microprocessors are based on the principle of store-program concept. In *store-program concept*, programs or instructions are sequentially stored in the memory locations that are to be executed.

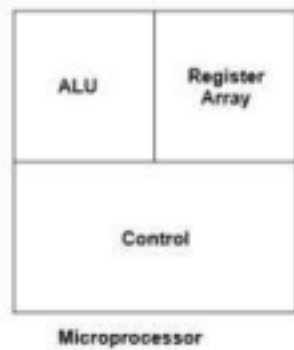


Fig: Functional Block Diagram of Microprocessor

1. ALU – Arithmetic and Logic Unit

The ALU carries out the arithmetic and logic operations on 8-bit words. It can perform arithmetic operations such as addition, subtraction and logical operations such as AND, OR and EX-OR.

The contents of the accumulator and the temporary register are the inputs to the ALU. The ALU result is then stored back in the accumulator. The ALU of a simple less powerful microprocessor such as Intel 8085 does not have multiplication and division instruction.

2. Register Array

Register is a storage unit within the microprocessor used to store the data, address of instruction of any program.

Microprocessor contains six general purpose registers it has 8-bit memory and registers are B, C, D, E, H and L register.

To hold 16-bit data a combination of two 8-bit registers can be used. The combination of two 8-bit registers is known as Register Pair (BC, DE and HL). These registers are used to store data temporarily during the execution of the program and are accessible to the user through instructions.

3. Control Unit

The timing and control unit acts as the brain of a computer. It controls all operations of the CPU. It also controls input, output and all other devices connected to the CPU. It generates signals which are required for all the operations to be performed by the CPU and required for the control of input/output and other devices connected to CPU. Such signals are also called timing signals because it has strict relation with clock signal (i.e., time) of the processor.

8 – BIT MICROPROCESSOR (8085): FEATURES AND ARCHITECTURE

Features:

- Intel 8085 is an 8-bit, NMOS microprocessor.
- It is a 40 pin I.C. package fabricated on a single LSI chip.
- The Intel 8085 uses a single +5Vd.c. supply for its operation.
- Its clock speed is about 3 MHz
- It is enclosed with 40 pins DIP (Dual in line package)
- The clock cycle is of 320 ns.
- The width of the data bus is 8 – bit and for address bus is 16- bits and hence can address up to $2^{16} = 65536$ bytes (64KB) memory locations.
- It supports external interrupt request.

8085 MICROPROCESSOR ARCHITECTURE

The 8085 microprocessor is an 8-bit processor available as a 40-pin IC package and uses +5 V for power. It can run at a maximum frequency of 3 MHz. Its data bus width is 8-bit and address bus width is 16-bit, thus it can address $2^{16} = 64$ KB of memory. The internal architecture of 8085 is shown in Fig.

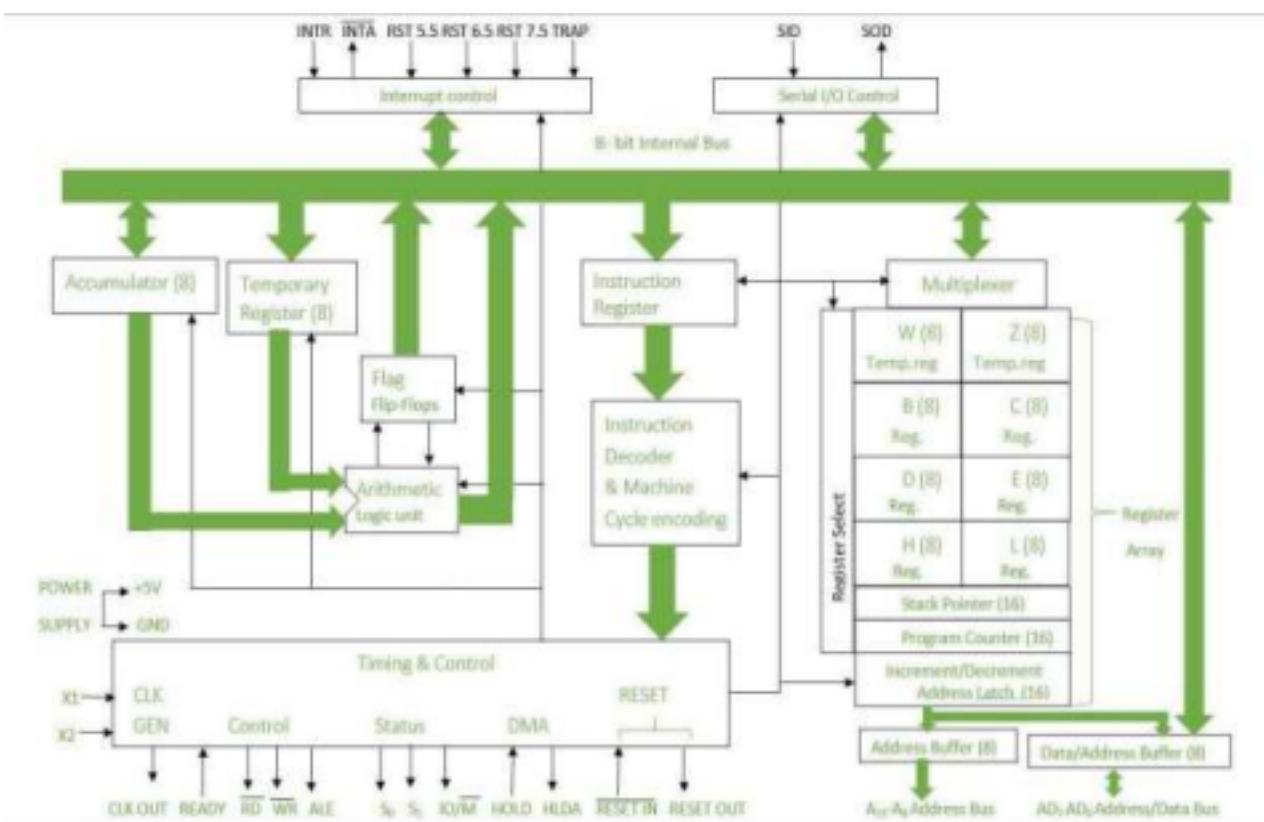


Fig. Architecture of

8085

It consists of three main sections:

- [1] Arithmetic and Logic Unit (ALU)
- [2] Timing & Control Unit

[3] Set of registers.

1. Arithmetic and Logic Unit

The ALU performs the actual numerical and logical operations such as

- Addition (ADD)
- Subtraction (SUB)
- Logical AND
- Logical OR
- Logical EX-OR.
- Increment (add 1)
- Decrement (Subtract 1)
- Left shift, Rotate left, Rotate Right.

2. Timing & Control Unit:

- It generates timing & control signals which are necessary for the execution of instructions.
- It controls data flow between CPU and peripherals including memory.
- It provides status, control and timing signals which are required for the operation of memory and I/O devices.
- It controls entire operations of the microprocessor and peripherals connected to it.
- Thus, it is seen that the control unit of the CPU acts as the brain of the computer system.

3. Registers:

Intel 8085 microprocessor has the following registers:

- i. One 8-bit Accumulator (ACC) i.e. register A
- ii. Six 8-bit general purpose registers. These are B, C, D, E, H & L
- iii. One 16-bit stack pointer, SP
- iv. One 16-bit program Counter, PC
- v. Instruction registers
- vi. Temporary register
- vii. Flag register

Accumulator (ACC):

- The accumulator (register A) is an 8-bit register associated with ALU.
- It is used to hold one of the operands of an arithmetic or logical operation.
- It serves as one input to the ALU. The other operand for an arithmetic or logical operation may be stored either in the memory or in one of the general-purpose registers.
- Final result of an arithmetic or logical operation is placed in the accumulator.

General Purpose Registers

- The 8085 microprocessor contains six 8-bit general purpose registers. They are B, C, D, E, H and L. • To hold 16-bit data a combination of two 8-bit registers can be employed which known as register pairs. • The valid register pairs in 8051 are B-C, D-E and H-L. the programmer cannot form the register pair by selecting any two registers of his choice.
- The H-L pair is used to act as memory pointer and for this purpose it holds the 16-bit address of a memory location.
- The general-purpose registers and the accumulator are accessible to programmer. He can store data in these registers during writing his program.

Program Counter (PC):

- It is a 16-bit special purpose register used to hold the memory address of the next instruction which is to be executed.
- It keeps the track of memory addresses of the instructions in the program while they are being executed. • The microprocessor increments the content of the program counter during the execution of an instruction so that it points to the address of the next instruction in the program at the end of the execution of an instruction.

Stack Pointer (SP):

- It is a 16-bit special purpose register.
- Stack is a sequence of memory location set aside by the programmer to store/retrieve the content of accumulator, flags, program counter and general-purpose registers during the execution of a program.
- Any portion of the memory can be used as stack. Since the stack works on LIFO (last in first out) principle, its operation is faster compared normal store/ retrieve of memory locations.
- The SP holds the address of the top element of data stored in the stack.
- The stack is defined and stack pointer is initialized by the programmer at the beginning of a program which needs stack operation. Stack is also used by the microprocessor. For example, it stores the contents of program counter when it jumps to a subroutine using CALL instruction.

Instruction Register:

- The instruction register holds the opcode (operation code or instruction code) of the instruction which is being decoded and executed.

Temporary Register:

- It is an 8-bit register associated with ALU. It holds data during an arithmetic/logical operation.
- It is used by microprocessor.
- It is not accessible to programmer.

Flags (Program Status Word i.e. PSW):

- The Intel 8085 microprocessor contains five flip-flops to serve as status flags.
- The flip-flops are set or reset according to the conditions which arises during an arithmetic or logical operation.
- The five status flags of Intel 8085 are:

- i. Carry flag (CS)
- ii. Parity Flag (P)
- iii. Auxiliary Carry Flag (AC)
- iv. Zero Flag (Z)
- v. Sign Flag (S)

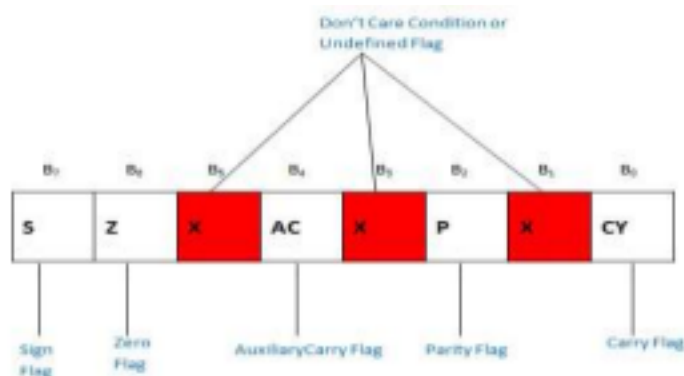


Fig. Flag Register

In fig above, five bits indicates the five status flags and three bits are undefined. “The combination of these 8-bits is called Program Status Word (PSW)”. PSW and the accumulator is treated as a 16-bit unit for stack operation.

Carry Flag (CS):

- After execution of an arithmetic instruction if carry is produced, the carry flag CS is set to 1. Otherwise it is 0.
- The carry flag is set or reset in case of addition as well as subtraction.
- After the addition of two 8-bit numbers, if the sum is larger than 8-bits, a carry is producing; and the carry flag is set to 1.
- In case of subtraction, if borrow occurs, the carry flag is set to 1.
- The carry flag holds carry out of the most significant bit resulting from the execution of an arithmetic operation.

Parity Flag (P):

- The parity flag P is set to 1, if the result of an arithmetic or logical operation contains even number of 1s.
- It is reset i.e. it is 0, if the result contains odd number of 1s.

Auxiliary Carry Flag (AC):

- The auxiliary carry flag AC hold carry out of the bit number 3 to the bit number 4 resulting from the execution of an arithmetic operation.
- The counting of bits starts from 0, and hence Bit No.3 is actually the fourth bit from the least significant bit.

Zero Flag (Z):

The zero-status flag Z is set to 1, if the result of an arithmetic or logical operation is Zero otherwise it is set to 0.

Sign Flag (S):

- The sign flag S is set to 1, if the result of an arithmetic or logical operation is negative. If the result is positive, the sign

flag is set to 0.

- The MSB is used as a sign bit.

Serial Input/Output Control

It controls the serial data communication by using Serial input data and Serial output data. Serial Input/Output control in the 8085 microprocessor refers to the communication of data between the microprocessor and external devices in a serial manner, i.e., one bit at a time.

The 8085 has a serial I/O port (SID/SOD) for serial communication. The SID pin is used for serial input and the SOD pin is used for serial output. The timing and control of serial communication is managed by the 8085's internal circuitry.

Data and Address bus:

- The Intel 8085 is an 8-bit microprocessor. Its data bus is 8-bit wide and hence, 8bit of data can be transmitted in parallel from or to the microprocessor.
- The Intel 8085 requires a 16-bit wide address bus as the memory addresses are of 16 bits.
- The 8 most significant bits of the address are transmitted by the address bus, a-bus (pins A8 to A15).
- The least significant bits of address are transmitted by address/ data bus, AD-bus (pins AD0 to AD7).
- The higher order address lines (A8 – A15) are unidirectional and the lower order lines (A0 – A7) are multiplexed (time-shared) with the eight data bits (D0 – D7) and hence, they are bidirectional.
- The address/ data bus transmits data and address at different moments. At a particular moment it transmits either data or address. Thus AD-bus operates in time shared mode.

Control Bus:

Control bus is various lines which have specific functions for coordinating and controlling microprocessor operations. The control bus carries control signals partly unidirectional and partly bidirectional. The following control and status signals are used by 8085 processor:

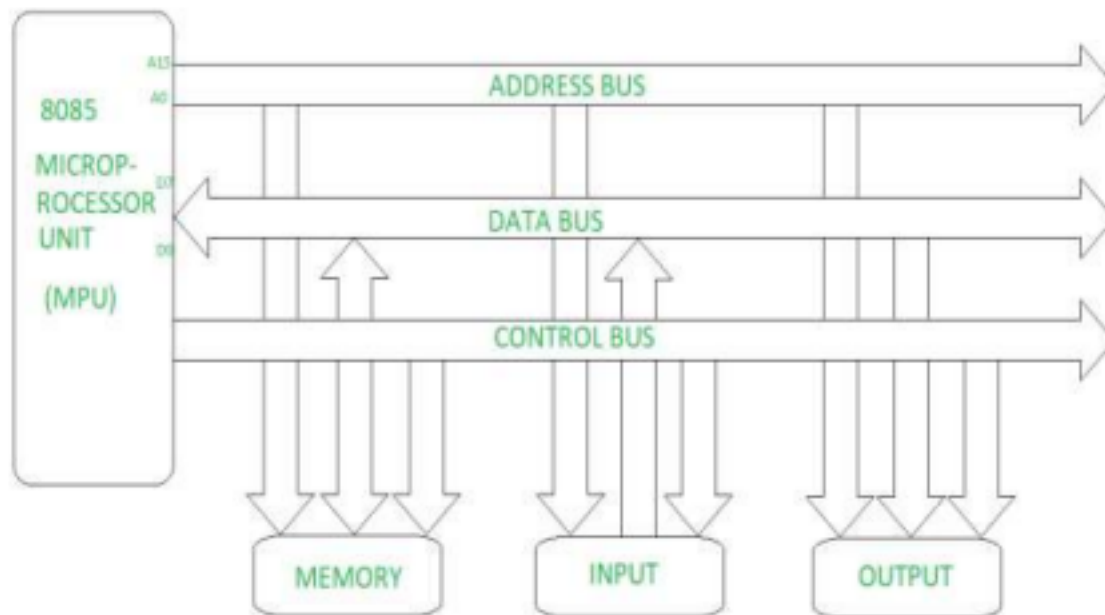
- ALE (output): Address Latch Enable is a pulse that is provided when an address appears on the AD0 – AD7 lines, after which it becomes 0.
- RD (active low output): The Read signal indicates that data are being read from the selected I/O or memory device and that they are available on the data bus.
- WR (active low output): The Write signal indicates that data on the data bus are to be written into a selected memory or I/O location.
- IO/M(output): It is a signal that distinguished between a memory operation and an I/O operation. When IO/M= 0 it is a memory operation and IO/M=1 it is an I/O operation.
- S1 and S0 (output): These are status signals used to specify the type of operation being performed; they are listed in Table 1.

Table 1 Status signals and associated operations

S1	S0	States
----	----	--------

0	0	Halt
0	1	Write
1	0	Read
1	1	Fetch

The schematic representation of the 8085-bus structure is as shown in Fig.



Bus organization system of 8085 Microprocessor

The microprocessor performs

primarily four operations:

I. Memory Read: Reads data (or instruction) from memory.

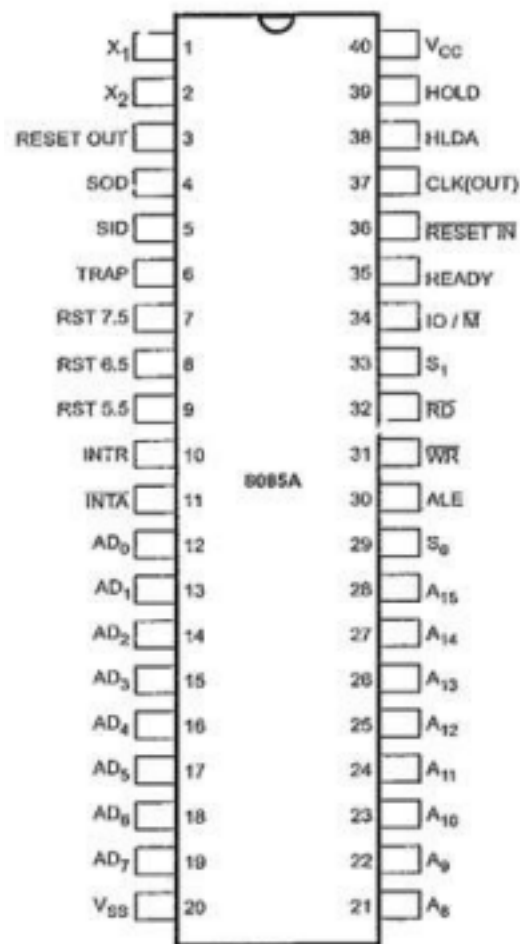
II. Memory Write: Writes data (or instruction) in to memory.

III. I/O Read: Accepts data from input device.

IV. I/O Write: Sends data to output device.

PIN CONFIGURATION OF 8085 MICROPROCESSOR

The pin configuration of 8085 microprocessor is shown below.



It consists of,

- Power Supply and Frequency Signals
- Data Bus and Address Bus
- Control and Status Signals
- Interrupt Signals
- Serial I/O Signals
- DMA Signal
- Reset Signals

1. Power Supply and Frequency Signals:

- V_{cc} : It requires a single +5 V power supply.
- V_{ss} : Ground reference.
- X_1 and X_2 : A tuned circuit like LC, RC or crystal is connected at these two. The internal clock generator divides oscillator frequency by 2, therefore, to operate a system at 3 MHz, the crystal of tuned circuit must have a frequency of 6 MHz.
- CLK OUT : This signal is used as a system clock for other devices. Its frequency is half the oscillator frequency.

2. Data Bus and Address Bus:

A) AD_0 to AD_7 : The 8 bit data bus ($D_0 - D_7$) is multiplexed with the lower half ($A_0 - A_7$) of the 16 bit address bus. During

first part of the machine cycle (T_1), lower 8 bits of memory address or I/O address appear on the bus. During remaining part of the machine cycle (T_2 and T_3) these lines are used as a bi-directional data bus.

B) A_8 to A_{15} : The upper half of the 16-bit address appears on the address lines A_8 to A_{15} . These lines are exclusively used for the most significant 8 bits of the 16-bit address lines.

3. Control and Status Signals:

A) ALE (Address Latch Enable) : AD_0 to AD_7 lines are multiplexed and the lower half of address ($A_0 - A_7$) is available only during T_1 of the machine cycle. This lower half of address is also necessary during T_2 and T_3 of machine cycle to access specific location in memory or I/O port. This means that the lower half of an address must be latched in T_1 of the machine cycle, so that it is available throughout the machine cycle. The latching of lower half of an address bus is done by using external latch and ALE signal from 8085 Pin Diagram.

B) RD and WR : These signals are basically used to control the direction of the data flow between processor and memory or I/O device/port. A low on RD indicates that the data must be read from the selected memory location or I/O port via data bus. A low on WR indicates that the data must be written into the selected memory location or I/O port via data bus.

C) IO/M, S_0 and S_1 : IO/M indicates whether I/O operation or memory operation is being carried out. S_1 and S_0 indicate the type of machine cycle in progress.

D) READY: It is used by the microprocessor to sense whether a peripheral is ready or not for data transfer. If not, the processor waits. It is thus used to synchronize slower peripherals to the microprocessor.

4. Interrupt Signals:

The 8085 Pin Diagram has five hardware interrupt signals: RST 5.5, RST 6.5, RST 7.5, TRAP and INTR. The microprocessor recognizes interrupt requests on these lines at the end of the current instruction execution.

The INTA (Interrupt Acknowledge) signal is used to indicate that the processor has acknowledged an INTR interrupt.

5. Serial I/O Signals:

A) SID (Serial I/P Data): This input signal is used to accept serial data bit by bit from the external device.

B) SOD (Serial O/P Data) : This is an output signal which enables the transmission of serial data bit by bit to the external device.

6. DMA Signal:

A) HOLD: This signal indicates that another master is requesting for the use of address bus, data bus and control bus.

B) HLDA: This active high signal is used to acknowledge HOLD request.

7. Reset Signals:

A) RESET IN: A low on this pin

- Sets the program counter to zero (0000H).
- Resets the interrupt enable and HLDA flip-flops.
- Tri-states the data bus, address bus and control bus.
- Affects the contents of processor's internal registers randomly.

On reset, the PC sets to 0000H which causes the 8085 Pin Diagram to execute the first instruction from address 0000H. For proper reset operation reset signal must be held low for at least 3 clock cycles. The power-on reset circuit can be used to ensure execution of first instruction from address 0000H.

B) RESET OUT: This active high signal indicates that processor is being reset. This signal is synchronized to the processor clock and it can be used to reset other devices connected in the system.

INSTRUCTION SET AND EXECUTION IN 8085

Based on the design of the ALU and decoding unit, the microprocessor manufacturer provides instruction set for every microprocessor. The instruction set consists of both machine code and mnemonics.

An instruction is a binary pattern designed inside a microprocessor to perform a specific function. The entire group of instructions that a microprocessor supports is called instruction set. Microprocessor instructions can be classified based on the parameters such functionality, length and operand addressing.

CLASSIFICATION BASED ON FUNCTIONALITY:

1. Data transfer group.
2. Arithmetic group.
3. Logical group.
4. Branching group
5. Machine control group.

1. Data transfer group:

This group of instructions copies data from source to destination. The content of the source is not altered. Following are the list of **Data Transfer Instructions**.

2. Arithmetic group: Instructions of this group perform operations like addition, subtraction, increment & decrement. One of the data used in arithmetic operation is stored in accumulator and the result is also stored in accumulator.

3. Logical group: Logical operations include AND, OR, EXOR, NOT. The operations like AND, OR and EXOR uses two operands, one is stored in accumulator and other

can be any register or memory location. The result is stored in accumulator. NOT operation requires single operand, which is stored in accumulator.

4. Branching group: Instructions in this group can be used to transfer program sequence from one memory location to another either conditionally or unconditionally.

5. Machine control group: Instruction in this group control execution of other instructions and control operations like interrupt, halt etc.

Classification based on length:

- One-byte instructions: Instruction having one byte in machine code. Examples are depicted in Table

Opcode	Operand	Machine code/Hex code
MOV	A, B	78
ADD	M	86

- Two-byte instructions: Instruction having two byte in machine code.

Examples are depicted in Table

Opcode	Operand	Machine code/Hex code	Byte description
MVI	A, 7FH	3E	First byte
		7F	Second byte
ADI	0FH	C6	First byte
		0F	Second byte

- Three-byte instructions: Instruction having three bytes in machine code. Examples are depicted in Table

Opcode	Operand	Machine code/Hex code	Byte description
JMP	9050H	C3	First byte
		50	Second byte
		90	Third byte
LDA	8850H	3A	First byte
		50	Second byte
		88	Third byte

Notation	Meaning	Notation	Meaning
M	Memory location pointed by HL register pair	OF	Opcode
r	8-bit register	MR	Memory Register
rp	16-bit register pair	MW	Memory Word
rs	Source register	BI	Bus Interface
rd	Destination register		
addr	16-bit address / 8-bit address		

1.8.1 Data Transfer Group

MVI r, data (8)		Operation : $r \leftarrow$ 8-bit data (byte)
Description : This instruction directly loads a specified register with an 8-bit data given in the instruction. The register <i>r</i> is an 8-bit general purpose register such as A, B, C, D, E, H, L.		
Example : MVI B, 60H ; This instruction will load 60H directly into the B register		
Addressing Mode : Immediate		Flags Affected : None
Machine Cycles : OF, MR		

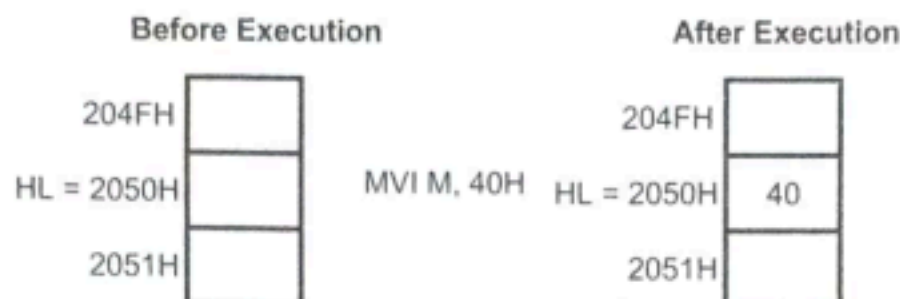
MVI M, data (8)**Operation :** $M \leftarrow \text{byte}$ or $(HL) \leftarrow \text{byte}$

By

Description : This instruction directly loads an 8-bit data given within the instruction memory location. The memory location is specified by the contents of HL register pair.

Example : H = 20H and L = 50H

MVI M, 40H ; This instruction will load 40H into memory whose address is 2050H.

**Addressing Mode :** Immediate and Indirect**Flags Affected :** None**Machine Cycles :** OF, MR, MW**MOV rd, rs****Operation :** $rd \leftarrow rs$

By

Description : This instruction copies data from the source register into destination register. rd and rs are general purpose registers such as A, B, C, D, E, H and L. The contents of the register remain unchanged after execution of the instruction.

Example : A = 20H

MOV B, A ; This instruction will copy the contents of register A (20H) into register B.

Addressing Mode : Register**Flags Affected :** None**Machine Cycles :** OF**MOV M, rs****Operation :** $(HL) \leftarrow rs$

By

Description : This instruction copies data from the source register into memory location by the HL register pair. The rs is an 8-bit general purpose register such as A, B, C, D, E, H and L.

Example : If HL = 2050H, B = 30H.

MOV M, B ; This instruction will copy the contents of B register (30H) into the memory whose address is specified by HL (2050H).

Addressing Mode : Indirect**Flags Affected :** None**Machine Cycles :** OF, MW

MOV rd, M	Operation : $rd \leftarrow (HL)$
Description : This instruction copies data from memory location whose address is in HL register pair into destination register. The contents of the memory location remain unchanged. The rd is an 8-bit general purpose register such as A, B, C, D, E, H and L.	
Example : HL = 2050H, contents at 2050H memory location = 40H MOV C, M; This instruction will copy the contents of memory location pointed by HL register pair (40H) into the C register.	
Addressing Mode : Indirect	Flags Affected : None
Machine Cycles : OF, MR	

LXI rp, data (16)	Operation : $rp \leftarrow \text{data (16)}$
Description : This instruction loads immediate 16 bit data specified within the instruction into register pair or stack pointer. The rp is 16-bit register pair such as BC, DE, HL or SP.	
Example : LXI B, 1020H; This instruction will load 10H into B register and 20H into C register.	
Addressing Mode : Immediate	Flags Affected : None
Machine Cycles : OF, MR, MR	

STA addr	Operation : $(addr) \leftarrow A$
Description : This instruction stores the contents of A register into the memory location whose address is directly specified within the instruction. The contents of A register remain unchanged.	
Example : A = 50H STA 2000H ; This instruction will store the contents of A register (50H) to memory location 2000H.	
Addressing Mode : Direct	Flags Affected : None
Machine Cycles : OF, MR, MR, MW	

LDA addr	Operation : $A \leftarrow (addr)$
Description : This instruction copies the contents of the memory location whose address is specified within the instruction into the accumulator. The contents of the memory location remain unchanged.	
Example : (2000H) = 30H LDA 2000H ; This instruction will copy the contents of memory location 2000H i.e. data at 2000H into the A register.	
Addressing Mode : Direct	Flags Affected : None
Machine Cycles : OF, MR, MR, MR	

SHLD addr	Operation : (addr) $\leftarrow$ L and (addr + 1) $\leftarrow$ H	Byte
Description : This instruction stores the contents of L register in the memory location within the instruction and contents of H register at address next to it. This instruction is used to store the contents of H and L registers directly into the memory. The contents of the H and L registers remain unchanged.		
Example : H = 30H, L = 60H SHLD 2500H ; This instruction will copy the contents of L register at address 2500H and contents of H register at address 2501H.		
Addressing Mode : Direct	Flags Affected : None	
Machine Cycles : OF, MR, MR, MW, MW		

LHLD addr	Operation : L $\leftarrow$ (addr), H $\leftarrow$ (addr + 1)	Byte
Description : This instruction copies the contents of the memory location given with instruction into the L register and the contents of the next memory location into the H register.		
Example : (2500H) = 30H, (2501H) = 60H LHLD 2500H ; This instruction will copy the contents of memory location 2500H i.e. data 30H into the L register and the contents at memory location 2501H i.e. data 60H into the H register.		
Addressing Mode : Direct	Flags Affected : None	
Machine Cycles : OF, MR, MR, MR, MR		

STAX rp	Operation : (rp) $\leftarrow$ A	By
Description : This instruction copies the contents of accumulator into the memory location address is specified by the specified register pair. The rp is BC or DE register pair. This pair is used as a memory pointer. The contents of the accumulator remain unchanged.		
Example : BC = 1020H, A = 50H STAX B ; This instruction will copy the contents of A register (50H) to the memory specified by BC register pair (1020H).		
Addressing Mode : Indirect	Flags Affected : None	
Machine Cycles : OF, MW		

LDAX rp	Operation : A $\leftarrow$ (rp)	Byte
Description : This instruction copies the contents of memory location whose address is specified by the register pair into the accumulator. The rp is BC or DE register pair. The register pair is used as a memory pointer.		
Example : DE = 2030H, (2030H) = 80H LDAX D ; This instruction will copy the contents of memory location specified by DE register pair (2030H) into the accumulator.		

Addressing Mode : Indirect	Flags Affected : None
Machine Cycles : OF, MR	

XCHG	Operation : $H \leftrightarrow D$ and $L \leftrightarrow E$	By
Description : This instruction exchanges the contents of the register H with that of D and L with that of E.		
Example : DE = 2040H, HL = 7080H XCHG ; This instruction will load the data into registers as follows H = 20H, L = 40H, and E = 80		
Addressing Mode : Register	Flags Affected : None	
Machine Cycles : OF		

1.8.2 Arithmetic Group

ADD r	Operation : $A \leftarrow A + r$	B
Description : This instruction adds the contents of the specified register to the contents of the accumulator and stores result in the accumulator. The r is 8-bit general purpose register A, B, C, D, E, H and L.		
Example : A = 20H, C = 30H. ADD C ; This instruction will add the contents of C register, i.e. data 30H to the contents of accumulator, i.e. data 20H and it will store the result 50H in the accumulator.		
Addressing Mode : Register	Flags Affected : All	
Machine Cycles : OF		

ADD M	Operation : $A \leftarrow A + (HL)$	E
Description : This instruction adds the contents of the memory location pointed by HL register pair to the contents of accumulator and stores result in the accumulator. The HL register is used as a memory pointer. This instruction affects all flags.		
Example : A = 20H, HL = 2050H, (2050H) = 10H ADD M ; This instruction will add the contents of memory location pointed by HL register pair i.e. data 10H to the contents of accumulator i.e. data 20H and it will store the result 30H in the accumulator.		
Addressing Mode : Register Indirect	Flags Affected : All	
Machine Cycles : OF, MR		

ADI data (8)	Operation : $A \leftarrow A + \text{data (8)}$	Bytes :
Description : This instruction adds the 8 bit data given within the instruction to the contents of accumulator and stores the result in the accumulator.		
Example : A = 50H ADI 70H ; This instruction will add 70H to the contents of the accumulator (50H) and it will store the result in the accumulator (C0H).		
Addressing Mode : Immediate	Flags Affected : All	
Machine Cycles : OF, MR		

ADC r	Operation : $A \leftarrow A + r + CY$	Bytes
Description : This instruction adds the contents of specified register to the content accumulator with carry. This means, if the carry flag is set by some previous operation, it adds and the contents of the specified register to the contents of accumulator, else it adds the content of the specified register only. The r is 8-bit general purpose register such as A, B, C, D, E, H, L.		
Example : Carry flag = 1, A = 50H, C = 20H ADC C ; This instruction will add the contents of C (20H) register to the contents of accumulator (50H) with carry (1) and it will store result, 71H ($50H + 20H + 1 = 71H$) in the accumulator.		
Addressing Mode : Register		Flags Affected : All
Machine Cycles : OF		

ADC M	Operation : $A \leftarrow A + M + CY$	Byte
Description : This instruction adds the contents of memory location pointed by HL register pair to the contents of accumulator with carry and stores the result in the accumulator. HL register pair is used as a memory pointer.		
Example : Carry flag = 1, HL = 2050H, A = 20H, (2050H) = 30H. ADC M ; This instruction will add the contents of memory location pointed by HL register 2050H, i.e. data 30H to the contents of accumulator, i.e. data 20H with carry flag (1). It will store the result (30+20+1=51H) in the accumulator.		
Addressing Mode : Register Indirect		Flags Affected : All
Machine Cycles : OF, MR		

ACI data (8)	Operation : $A \leftarrow A + \text{data (8)} + CY$	Bytes :
Description : This instruction adds 8 bit data given within the instruction to the contents of accumulator with carry and stores result in the accumulator.		

Example : A = 30H, Carry flag = 1

ACI 20H ; This instruction will add 20H to the contents of accumulator, i.e. data carry (1) and stores the result, 51H ($30 + 20 + 1 = 51H$) in the accumulator.

Addressing Mode : Immediate

Flags Affected : All

Machine Cycles : OF, MR

DAD rp

Operation : $HL \leftarrow HL + rp$

Description : This instruction adds the contents of the specified register pair to the contents of the HL register pair and stores the result in the HL register pair. The rp is 16-bit register pair such as BC, DE, HL or stack pointer. Only higher order register is to be specified for register pair within the instruction.

Example : DE = 1020H, HL = 2050H

DAD D ; This instruction will add the contents of DE register pair, 1020H to the contents of HL register pair, 2050H. It will store the result, 3070H in the HL register pair.

Addressing Mode : Register

Flags Affected : CY

Machine Cycles : OF, BI, BI

SUB r

Operation : $A \leftarrow A - r$

Description : This instruction subtracts the contents of the specified register from the contents of the accumulator and stores the result in the accumulator. The register r is 8-bit general purpose register such as A, B, C, D, E, H and L.

Example : A = 50H, B = 30H.

SUB B ; This instruction will subtract the contents of B register (30H) from the contents of accumulator (50H) and stores the result (20H) in the accumulator.

Addressing Mode : Register

Flags Affected : All

Machine Cycles : OF

SUB M

Operation : $A \leftarrow A - M$

Description : This instruction subtracts the contents of the memory location pointed by the register pair from the contents of accumulator and stores the result in the accumulator. The register pair is used as a memory pointer.

Example : HL = 1020H, A = 50H, (1020H) = 10H

SUB M ; This instruction will subtract the contents of memory location pointed by HL register pair, 1020H, i.e. data 10H from the contents of accumulator, i.e. data 50H and stores the result in accumulator.

Addressing Mode : Register Indirect

Flags Affected : All

Machine Cycles : OF, MR

Example : Carry flag = 1, A = 50H

SBI 20H ; This instruction will subtract 20H and the carry flag (1) from the contents of the accumulator (50H). It will store the result (50H - 20H - 1 = 2FH) in the accumulator.

Addressing Mode : Immediate

Flags Affected : All

Machine Cycles : OF, MR

DAA

Operation : $A(\text{BCD}) \leftarrow A(\text{Binary})$

Description : This instruction adjusts accumulator to packed BCD (Binary Coded Decimal) by adding two BCD numbers.

Instruction works as follows :

1. If the value of the low - order four bits (D_3-D_0) in the accumulator is greater than 9, the auxiliary carry flag is set, the instruction adds 6 (06) to the low-order four bits.
2. If the value of the high-order four bits ($D_7 - D_4$) in the accumulator is greater than 9, the carry flag is set, the instruction adds 6 (60) to the high-order four bits.

Example : If, A = 0011 1001 = 39 BCD and C = 0001 0010 = 12 BCD then

ADD C ; Gives A = 0100 1011 = 4BH

DAA ; adds 0110 because (lower nibble) 1011 > 9,
; A = 0101 0001 = 51 BCD

If A = 1001 0110 = 96 BCD and D = 0000 0111 = 07 BCD then

ADD D ; Gives A = 1001 1101 = 9DH

DAA ; adds 0110 because (lower nibble) 1101 > 9,
; A = 1010 0011 = A3H,
; so adds 0110 0000 because (higher nibble) 1010 > 9,
; A = 0000 0011 = 03 BCD, CF = 1.

Addressing Mode : Implied

Flags Affected : All

Machine Cycles : OF

INR r

Operation : $r \leftarrow r + 1$

Description : This instruction increments the contents of specified register by 1. The result is stored in the same register. The register r is 8-bit general purpose register such as A, B, C, D, E, H and L.

Example : B = 10H

INR B ; This instruction will increment the contents of B register (10H) by one and store the result (10+1 = 11H) in the same i.e. B register.

Addressing Mode : Register

Flags Affected : S, Z, P, A, C

Machine Cycles : OF







































INSTRUCTION EXECUTION AND TIMING DIAGRAM

Each instruction in 8085 microprocessor consists of two part- operation code (opcode) and operand. The opcode is a command such as ADD and the operand is an object to be operated on, such as a byte or the content of a register.

Instruction Cycle: The time taken by the processor to complete the execution of an instruction. An instruction cycle consists of one to six machine cycles.

Machine Cycle: The time required to complete one operation; accessing either the memory or I/O device. A machine cycle consists of three to six T-states.

T-State: Time corresponding to one clock period. It is the basic unit to calculate execution of instructions or programs in a processor.

To execute a program, 8085 performs various operations as:

- Opcode fetch
- Operand fetch
- Memory read/write
- I/O read/write

External communication functions are:

- Memory read/write

- I/O read/write
- Interrupt request acknowledge

1. Opcode Fetch Machine Cycle:

It is the first step in the execution of any instruction. The timing diagram of this cycle is given in Fig.

The following points explain the various operations that take place and the signals that are changed during the execution of opcode fetch machine cycle:

T1 clock cycle

- The content of PC is placed in the address bus; AD0 - AD7 lines contains lower bit address and A8 – A15 contains higher bit address.
- IO/M signal is low indicating that a memory location is being accessed. S1 and S0 also changed to the levels as indicated in Table 1.
- ALE is high, indicates that multiplexed AD0 –AD7 act as lower order bus.

T2 clock cycle

- Multiplexed address bus is now changed to data bus.
- The RD signal is made low by the processor. This signal makes the memory device load the data bus with the contents of the location addressed by the processor.

T3 clock cycle

- The opcode available on the data bus is read by the processor and moved to the instruction register.
- The RD signal is deactivated by making it logic1.

T4 clock cycle

- The processor decodes the instruction in the instruction register and generate the necessary control signals to execute the instruction. Based on the instruction further operations such as fetching, writing into memory etc takes place.

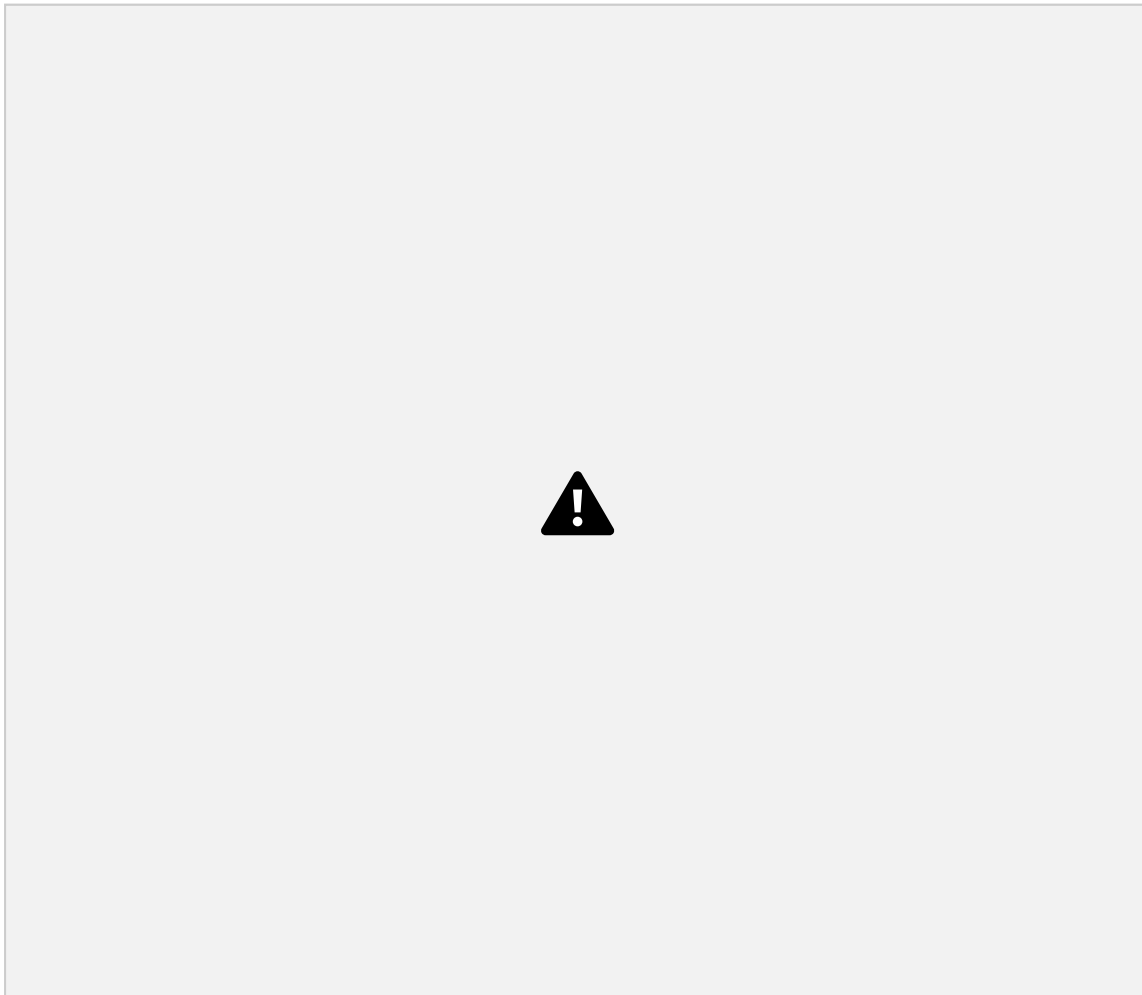


Fig.

Timing diagram for opcode fetch cycle

2. Memory Read Machine Cycle:

The memory read cycle is executed by the processor to read a data byte from memory. The machine cycle is exactly same to opcode fetch except: a) It has three T-states b) The S0 signal is set to 0. The timing diagram of this cycle is given in Fig. below

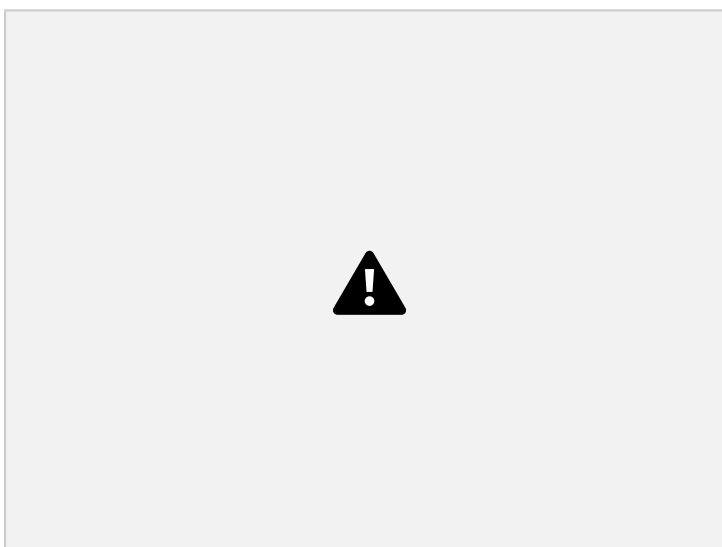


Fig. Timing diagram for memory read machine cycle

3. Memory Write Machine Cycle:

The memory write cycle is executed by the processor to write a data byte in a memory location. The processor takes three T-states and WR signal is made low. The timing diagram of this cycle is given in Fig.

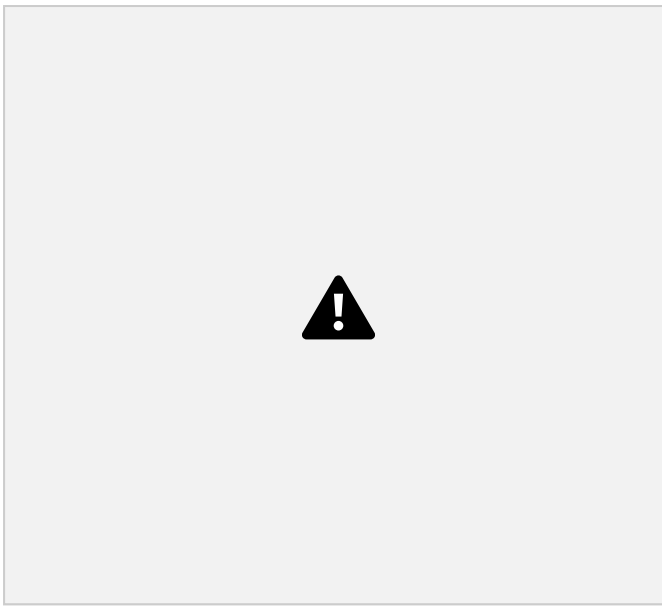


Fig. Timing diagram of memory write machine cycle

4. I/O Read Cycle:

The I/O read cycle is executed by the processor to read a data byte from I/O port or from peripheral, which is I/O mapped in the system. The 8-bit port address is placed both in the lower and higher order address bus. The processor takes three T-states to execute this machine cycle. The timing diagram of this cycle is given in Fig.

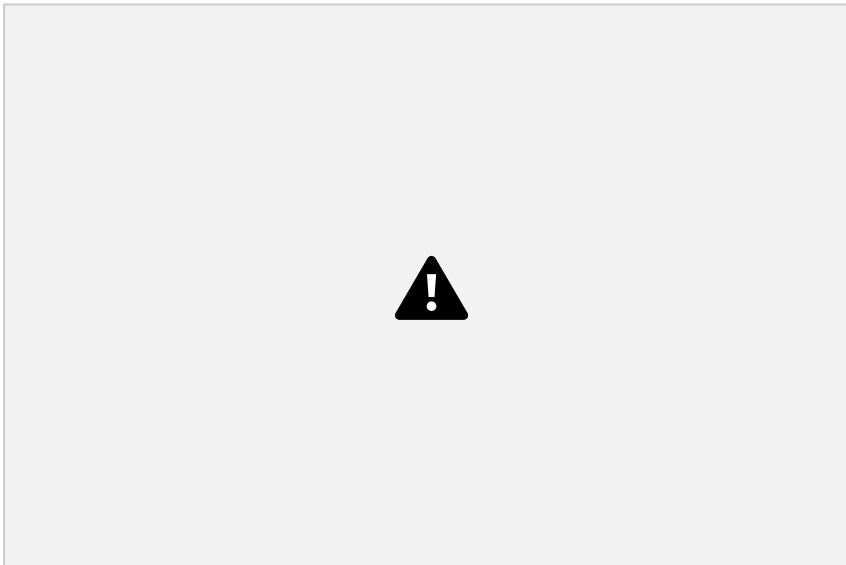


Fig. Timing diagram I/O read machine cycle

5. I/O Write Cycle:

The I/O write cycle is executed by the processor to write a data byte to I/O port or to a peripheral, which is I/O mapped in the system. The processor takes three T-states to execute this machine cycle. The timing diagram of this cycle is given in Fig.

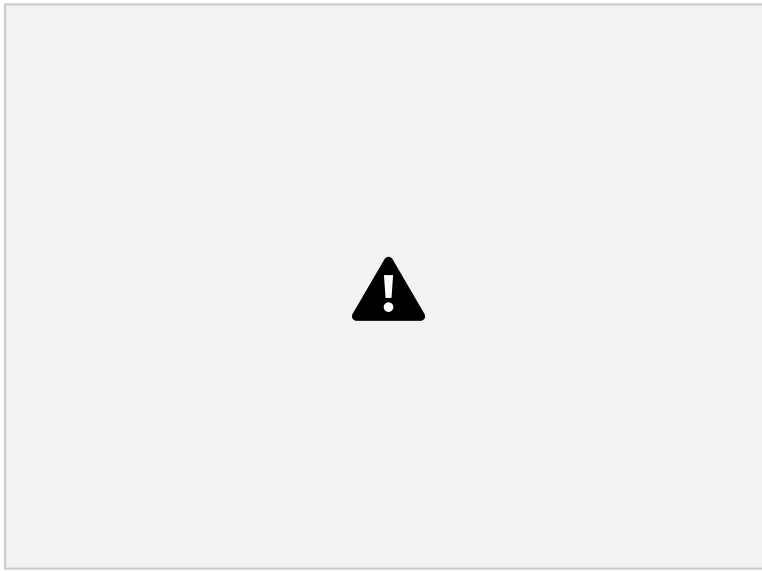


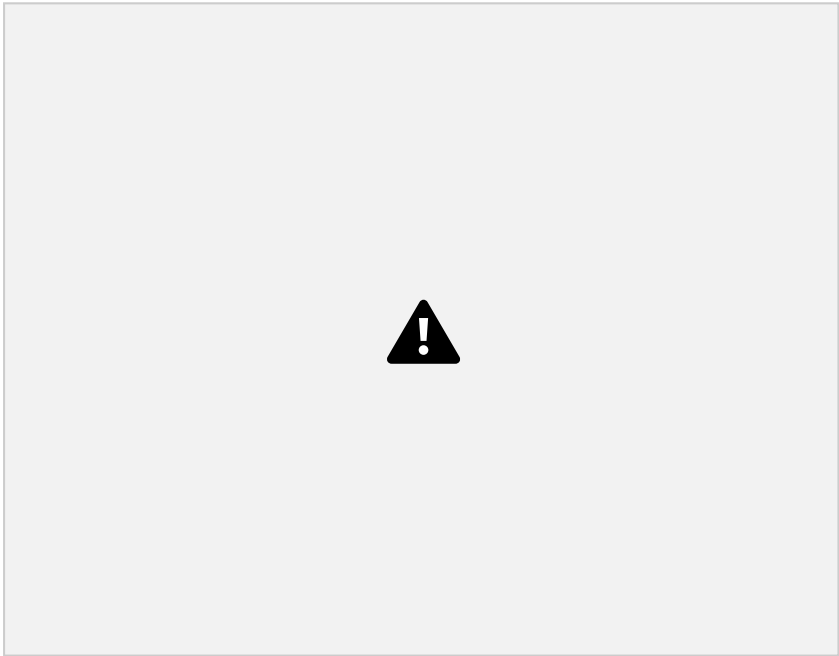
Fig. Timing diagram I/O write machine cycle
Ex: Timing diagram for IN 80H.

The instruction and the corresponding codes and memory locations are given in Table. Table
IN instruction

Address	Mnemonics	Opcode
800F	IN80H	DB
8010		80

- During the first machine cycle, the opcode DB is fetched from the memory, placed in the instruction register and decoded.
- During second machine cycle, the port address 80H is read from the next memory location.
- During the third machine cycle, the address 80H is placed in the address bus and the data read from that port address is placed in the accumulator.

The timing diagram is shown in Fig. Below



ASSEMBLY LANGUAGE PROGRAMS USING 8085 MICROPROCESSORS 1.

ADDITION OF TWO 8 BIT NUMBERS USING 8085 MICROPROCESSORS

FLOWCHART



ALGORITHM:

1. Initialize memory pointer to data location.
2. Get the first number from memory in accumulator.
3. Get the second number and add it to the accumulator.
4. Store the answer at another memory location.

PROGRAM:

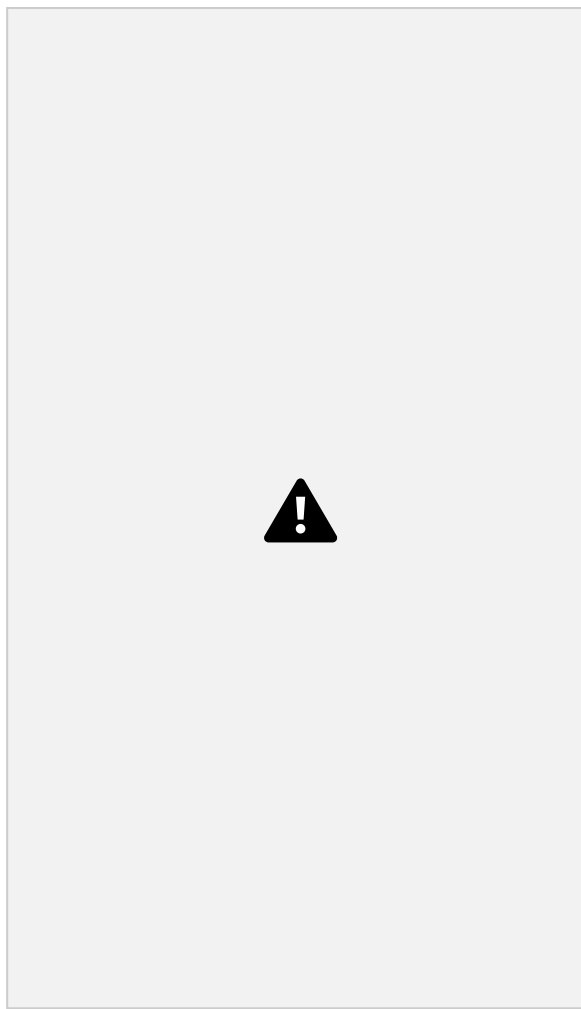
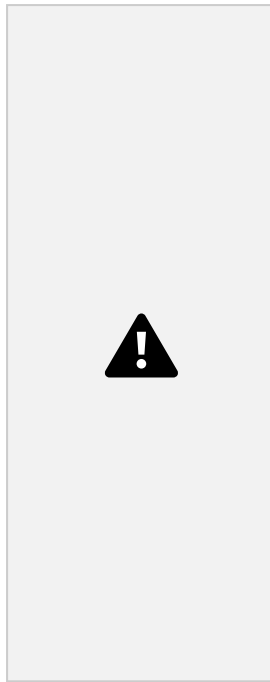
ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND	COMMENT
4100		START	MVI	C,00	Clear Creg.
4101					
4102			LXI	H,4200	Initialize HLreg.to 4500

4103					
4104					
4105			MOV	A,M	Transfer first data to accumulator
4106			INX	H	Increment HL reg.to Point next memory Location.
4107			ADD	M	Add first number to acc. Content.
4108			JNC	L1	Jump to location if result does not yield carry.
4109					
410A					
410B			INR	C	Increment C reg.
410C		L1	INX	H	Increment HL reg.to point next memory Location.
410D			MOV	M,A	Transfer the result from acc. To memory.
410E			INX	H	Increment HL reg.to Point next memory Location.
410F			MOV	M,C	Move carry to memory
4110			HLT		Stop the program

Thus the two 8 bit numbers stored at 4200 & 4201 are added and the result is stored at 4202 & 4203.

2. 8-BIT DATA SUBTRACTION

FLOWCHART



ALGORITHM:

1. Initialize memory pointer to data location.
2. Get the first number from memory in accumulator.
3. Get the second number and subtract from the accumulator.
4. If the result yields a borrow, the content of the acc. Is complemented and 01H is added to it (2's complement). A register is cleared and the content of that reg. is incremented in case there is a borrow. If there is no borrow, the content of the acc. is directly taken as the result.
5. Store the answer at next memory location.

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND	COMMENT
4100		START	MVI	C,00	Clear Creg.
4102					
4102			LXI	H,4200	Initialize HL reg.to 4200
4103					
4104					
4105			MOV	A,M	Transfer first data to accumulator
4106			INX	H	Increment HL reg.to Point next memory

					Location.
4107			SUB	M	Subtract first number from acc. Content.
4108			JNC	L1	Jump to location if result does not yield borrow.
4109					
410A					
410B			INR	C	Increment Creg.
410C			CMA		Complement the Acc. Content
410D			ADI	01H	Add 01H to content of acc.
410E					
410F		L1	INX	H	Increment HL reg.to Point next memory Location.
4110			MOV	M,A	Transfer the result from acc. to memory.
4111			INX	H	Increment HLreg.to point next memory Location.
4112			MOV	M,C	Move carry to memory.
4113			HLT		Stop the program

Thus the 8 bit numbers stored at 4200 & 4201 are subtracted and the result is stored at 4202 & 4203.

3. 8-BIT AND OPERATION

FLOWCHART

**PROGRAM:**

ADDRES S	OPCODE		MNEMONICS		COMMENT
4100			LXI H	4200	HL←4200H(source)
4101					Low byte of 4200H
4102					High byte of 4200H
4103			MOV	A,M	Move first Number to Accumulator
4104			INX	H	Point HL to 4201H
4105			ANA	M	A=A AND 4201H
4106			INX	H	Increment HL→HL now points to 4202H
4107			MOV	M,A	Store result at 4202H
4108			HLT		

Thus, the 8-bit numbers stored at 4200H and 4201H are ANDed, and the 8-bit result is stored at 4202H.

4. 8-BIT OR OPERATION**FLOWCHART**



PROGRAM:

ADDRES S	OPCODE		MNEMONICS		COMMENT
4100			LXI H	4200	HL←4200H (source)
4101					Low byte of 4200H
4102					High byte of 4200H
4103			MOVA,M		Move first Number to Accumulator
4104			INXH		Point HL to 4201H
4105			ORAM		A=A OR 4201H
4106			INXH		Increment HL→HL now points to 4202H
4107			MOVM,A		Store result at 4202H
4108			HLT		

Thus, the 8-bit numbers stored at 4200H and 4201H are ORed, and the 8-bit result is stored at 4202H.

5. 8-BIT INCREMENT OPERATION

FLOWCHART



PROGRAM:

ADDRES S	OPCODE		MNEMONICS		COMMENT
4100			LXI H,4200H	4200	Load Register Pair Immediate (HL).
4101					Loads L with 00H
4102					Loads H with 42H, HL = 4200H.
4103			MOVA,M		Move from memory (pointed by HL) to accumulator
4104			INRA		Increment Accumulator. A←A +1
4105			MOVM,A		Move accumulator to memory (pointed by HL).

Thus, the 8-bit number stored at 4200H and Incremented, and the 8-bit result is stored at 4201H.

6. 8-BIT DECREMENT OPERATION

FLOWCHART



PROGRAM:

ADDRESS	OPCODE		MNEMONICS		COMMENT
4100			LXI H,4200H	4200	Load Register Pair Immediate (HL).
4101					Loads L with 00H
4102					Loads H with 42H, HL = 4200H.
4103			MOV	A,M	Move from memory (pointed by HL) to accumulator
4104			DCR	A	Decrement Accumulator. A←A -1
4105			MOV	M,A	Move accumulator to memory (pointed by HL).

Thus, the 8-bit number stored at 4200H and decremented, and the 8-bit result is stored at 4201H.

7. DEMONSTRATE DATATRANSFER BETWEEN MEMORY AND REGISTERS

FLOWCHART



ALGORITHM:

1. Initialize HL register pair with the source address 4200H.
2. Initialize DE register pair with the destination address 4300H.
3. Initialize register B with the count 10H(i.e.,16 in decimal).
4. Start loop:
 - Copy the data from the memory location pointed by HL to accumulator A.
 - Store the accumulator content into the memory location pointed by DE.
 - Increment HL and DE to point to the next source and destination locations.
 - Decrement B (the counter).
 - If $B \neq 0$, repeat the loop.
6. Stop execution using the HLT instruction

PROGRAM:

ADDRES S	OPCODE		MNEMONICS		COMMENT
4100			LXI H	4200	HL←4200H(source)
4101					Low byte of 4200H
4102					High byte of 4200H
4103			LXI D	4300	DE←4300H(destination)
4104					Low byte of 4300H
4105					High byte of 4300H
4106			MVI B	10	B← 10H (count = 16)

4107					Immediate data
4108			MOVA, M		$A \leftarrow [HL]$
4109			STAXD		$[DE] \leftarrow A$
410A			INXH		$HL \leftarrow HL+1$
410B			INXD		$DE \leftarrow DE+1$
410C			DCRB		$B \leftarrow B-1$
410D			JNZ	0840	Jump to 4008H if B $\neq$ 0
410E					Low byte of jump address
410F					High byte of jump address
4110			HLT		Halt execution

Thus, the data from source (4200H) is transferred to the destination (4300H) successfully.

UNIT – 2 8086 MICROPROCESSOR ARCHITECTURE AND PROGRAMMING

16-bit Microprocessor (8086) Features and Architecture:

Features Of 8086 Microprocessor:

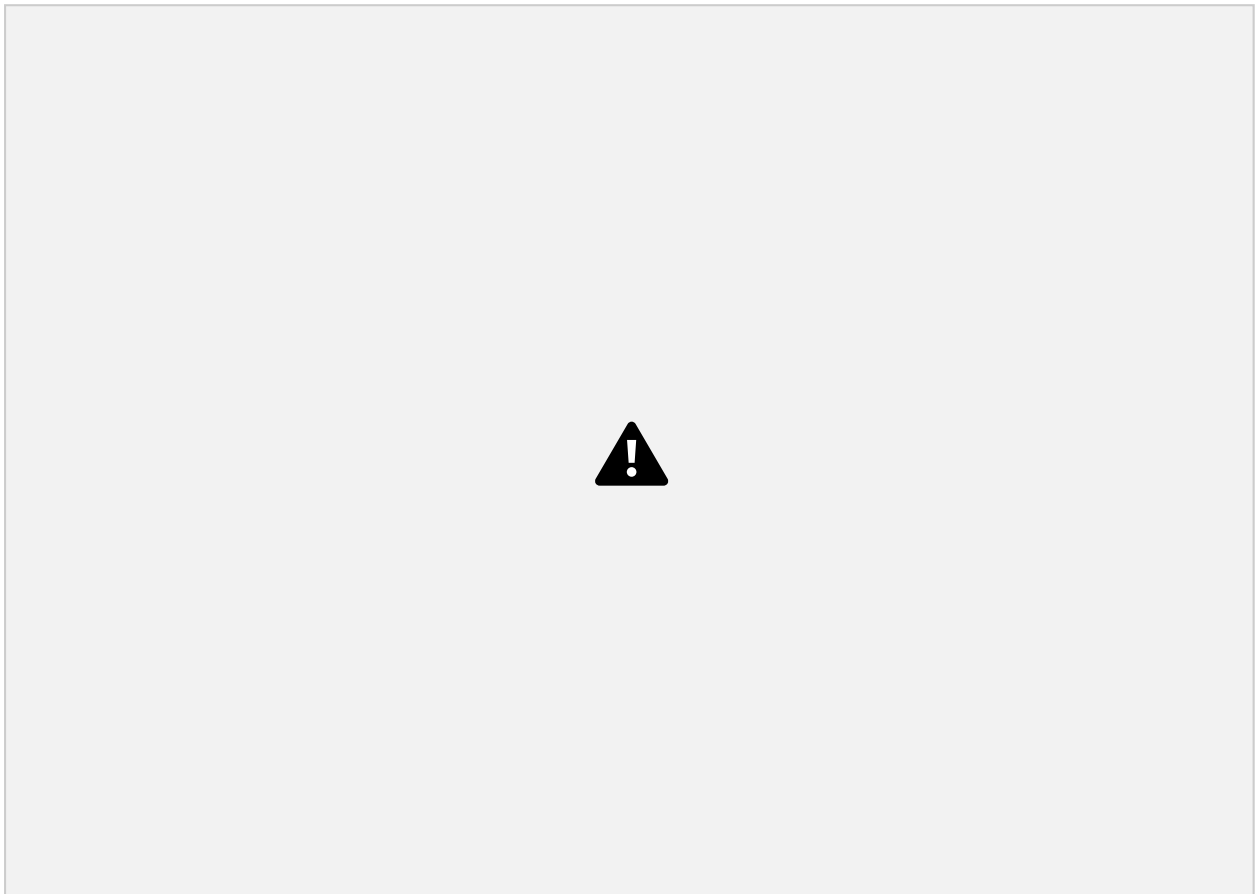
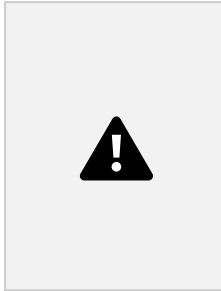
- 8086 Microprocessor is a 16-bit μ p.
- 8086 has a 20-bit address bus can access up to 2^{20} memory locations (1 MB). • It can support up to 64K I/O ports.
- It provides 14, 16 -bit registers.
- Word size is 16 bits and double word size is 4 bytes.
- It has multiplexed address and data bus AD0- AD15 and A16 – A19.
- It requires single phase clock with 33% duty cycle to provide internal timing. • 8086 is designed to operate in two modes, Minimum and Maximum.
- It can pre fetches up to 6 instruction bytes from memory and queues them in order to speed up instruction execution.
- It requires +5V power supply.
- A 40-pin dual in line package.
- Address ranges from 00000H to FFFFFH
- Memory is byte addressable - Every byte has a separate address.

Internal Architecture of 8086:

8086 has two blocks BIU and EU. The BIU handles all transactions of data and addresses on

the buses for EU. The BIU performs all bus operations such as instruction fetching, reading and writing operands for memory and calculating the addresses of the memory operands. The instruction bytes are transferred to the instruction queue. EU executes instructions from the instruction system byte queue.

Both units operate asynchronously to give the 8086 an overlapping instruction fetch and execution mechanism which is called as Pipelining. This results in efficient use of the system bus and system performance. BIU contains Instruction queue, Segment registers, Instruction pointer, and Address adder. EU contains Control circuitry, Instruction decoder, ALU, Pointer and Index register, Flag register.



1. Bus Interface Unit (BIU)

The BIU handles all data, address, and control signal transfers between the microprocessor and external devices.

Functions:

- Fetches instructions from memory.
- Calculates physical addresses using segment and offset.
- Queues instructions for the Execution Unit.

Components:

- **Segment Registers:**

- **CS** (Code Segment) – Stores code segment address.
- **DS** (Data Segment) – Stores data segment address.
- **SS** (Stack Segment) – Stack operations.
- **ES** (Extra Segment) – Additional data storage.

- **Instruction Pointer (IP)** – Holds offset of the next instruction.

- **Instruction Queue** – 6-byte FIFO queue for pre fetching instructions.

2. Execution Unit (EU)

The EU executes instructions fetched by the BIU.

Functions:

- Decodes and executes instructions.
- Controls ALU operations.
- Manages flag register.

Components:

- **ALU (Arithmetic Logic Unit)** – Performs arithmetic and logic operations. • **General**

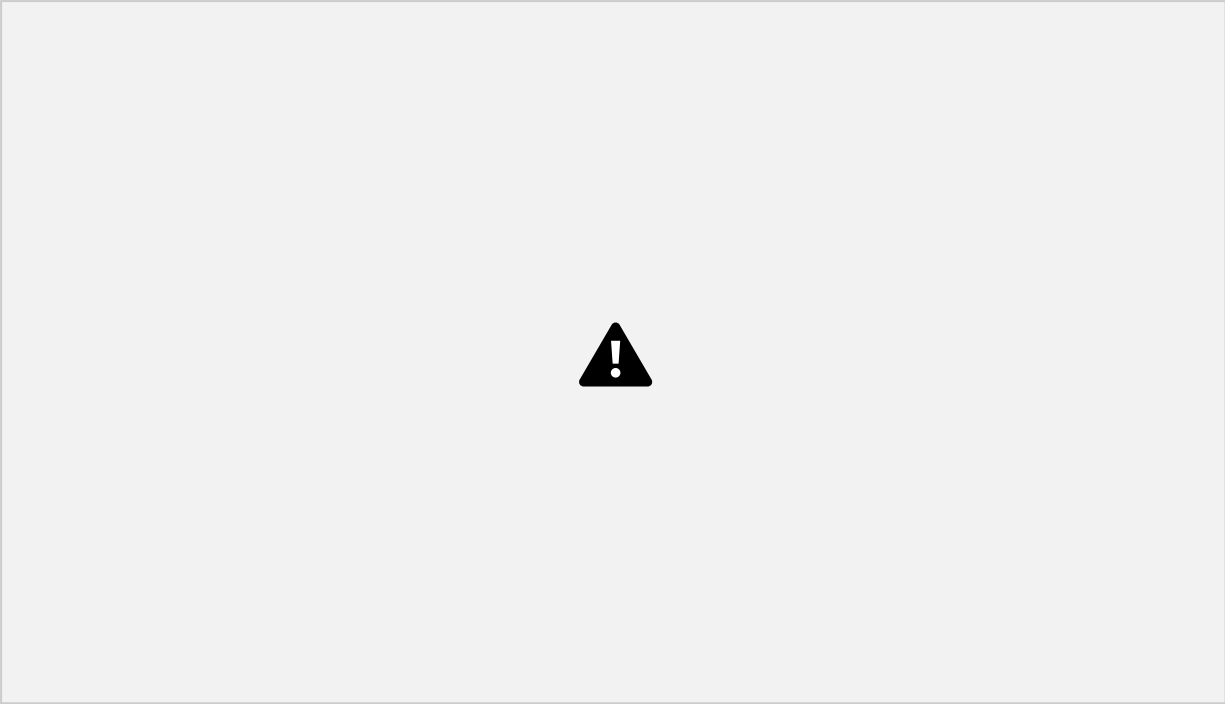
Purpose Registers: AX, BX, CX, DX (each can be split into high and low bytes).

- **Pointer and Index Registers:** SP, BP, SI, DI.

- **Flag Register** – Holds status flags (e.g., Zero, Carry, Sign, Overflow). • **Control Unit** – Directs operation flow.

EXECUTION UNIT

- Decodes instructions fetched by the BIU
- Generate control signals,
- Executes instructions.



The main parts are:

- Control Circuitry
- Instruction decoder
- ALU



AX Accumulator

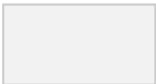
AH	AL
BH	BL
CH	CL
DH	DL
SP	
BP	
SI	
DI	

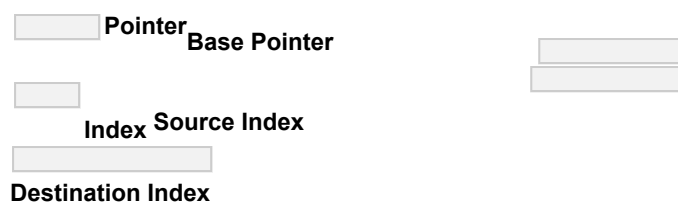
BX Base

CX Count

DX Data

Stack Pointer





Register Purpose

AX Word multiply, word divide, word I/O

AL	Byte multiply, byte divide, byte I/O, decimal arithmetic
AH	Byte multiply, byte divide
BX	Store address information
CX	String operation, loops
CL	Variable shift and rotate
DX	Word multiply, word divide, indirect I/O (Used to hold I/O address during I/O instructions. If the result is more than 16-bits, the lower order 16-bits are stored in accumulator and higher order 16-bits are stored in DX register)

9

Pointer and Index Registers

- Used to keep offset addresses.
- Used in various forms of memory addressing.
- In the case of SP and BP the default reference to form a physical address is the Stack Segment (SS- will be discussed under the BIU)
- The index registers (SI & DI) and the BX generally defaults to the Data segment register (DS).

SP: Stack pointer

- Used with SS to access the stack segment

BP: Base Pointer

- Primarily used to access data on the stack
- Can be used to access data in other segments

SI: Source Index register

- is required for some string operations

– When string operations are performed, the SI register points to memory locations in the data segment which is addressed by the DS register. Thus, SI is associated with the DS in string operations.

DI: Destination Index register

– is also required for some string operations.

– When string operations are performed, the DI register points to memory locations in the data segment which is addressed by the ES register. Thus, DI is associated with the ES in string operations.

– The SI and the DI registers may also be used to access data.

Flag Register

The 8086-flag **register** is a **16-bit register** used to indicate the current status of the processor and to control certain operations.

It has **two types of flags**:

1. **Status Flags** – Indicate the result of arithmetic or logic operations.

2. **Control Flags** – Control the processor's operation.

1. Status Flags

Flag Meaning Set When

CF (Carry Flag) Indicates carry/borrow in arithmetic
Carry out from MSB or borrow into MSB

PF (Parity Flag) Indicates parity of result Set if number of 1's in LSB is even

AF (Auxiliary Carry Flag)

For BCD operations Carry from bit 3 to bit 4

ZF (Zero Flag) Shows if result is zero Set if result = 0 **SF** (Sign Flag) Shows sign of result

Set if MSB = 1 (negative) **OF** (Overflow Flag) Indicates signed overflow Set if signed result exceeds range

2. Control Flags

Flag Meaning Use

TF (Trap Flag) Single-step mode Debugging

IF (Interrupt Flag) Enables interrupts Set to allow maskable interrupts

DF (Direction Flag)

String operation direction

0 = increment, 1 = decrement

SEGMENTED MEMORY MODEL AND PIN DIAGRAM

Memory Segmentation

Segmentation divides the main memory of a computer into logical segments to enhance execution speed, allowing the processor to efficiently fetch and execute data. The Intel 8086 microprocessor utilizes four segment registers to manage memory segments of 64KB each within a 1MB addressable space, providing advantages like improved memory management and data sharing.

The 8086 microprocessor uses four segment registers to manage memory:

- **Code Segment (CS):** Stores the executable program instructions.
- **Data Segment (DS):** Holds data required during program execution.
- **Stack Segment (SS):** Manages stack-related operations.
- **Extra Segment (ES):** Provides additional space for data overflow or auxiliary information.

Each segment is defined by a Segment Address (16-bit) and an Offset Address (16-bit). The physical address is calculated using the formula:

$$\text{Physical Address} = (\text{Segment Address} \times 10H) + \text{Offset Address}$$

For example, if the segment address is 1356H and the offset address is 2E20H: $PA = (1356H \times 10H) + 2E20H = 16380H$

Types of Segmentation

1. Overlapping Segmentation: Two segments share some memory space, meaning their memory ranges overlap.
2. Non-Overlapping Segmentation: Segments are distinct and do not share any memory space.

Rules of Segmentation

- The starting address of a segment must be divisible by 16.
- The size of a segment can range from 16 bytes to a maximum of 64 KB.

Advantages of Memory Segmentation

- **Efficient Memory Management:** Simplifies handling of large memory spaces.
- **Extended Addressing Capability:** Allows access to 1 MB of memory using 16-bit registers.
- **Logical Organization:** Separates code, data, and stack for better program structure.
- **Support for Large Programs:** Enables programs exceeding 64 KB by dividing them into multiple segments.
- **Data Sharing:** Facilitates easy sharing of data between processes.

Pin Diagram 8086 Microprocessor

8086 runs into 2 modes

- 1) Minimum mode
- 2) Maximum mode

The following pins are important in both minimum & maximum mode

1) **CLK (pin no-19)**

- The maximum clock frequency 5-10MHZ

- Provides basics timing for processor & bus controller.
- It is symmetric square wave with 33% duty cycle.

2) Vcc (pin no-40)

- Provide +5v power supply pin

3) GND (pin no-1,20)

- Ground for internal circuit

) RESET (pin no-21)

- It is a system reset
- When this signal goes high, processor enter into reset state & terminate the current activity & start execution from FFFF0H
- Active high signal -  for at least 4 clock cycle.

5) AD0-AD15

- These lines are multiplexed, bi-directional address / data bus
- 8086 has 20-bit address bus(A0-A19)
- 16-bit data bus
- During t1 cycle  carry lower order 16-bit address
- T2,t3,t4 -  carry 16 bit data
- So AD0-AD7 lines carry lower order byte of data& AD8 –AD15 carry high order byte of data

6) A19/ s6, A18/s5, A17/s4 , A16/s3

- These lines are multiplexed address & status lines
- Upper 4 bits address sent on this line for memory operation
- During I/O operation these lines are low
- During memory & I/O operation status information is available on these lines in T2,T4 cycles. - status of interrupt flag indicated by S5

S4 S3 segment Register

0 0 alternate data

0 1 stack

1 0 code

1 1 Data



7) BHE /S7 (pin no- 34) - ☐ bus high enable / status

During t1 state BHE signal is used to enable data on most significant half of data bus pins

D15- D8...S7 status information available during t2,t3,t4 status - ☐ remain high

BHE A0 word /byte access

0 0 whole word from even address 0 1 upper byte from / to odd address 1 0 lower byte from /
to even address 1 1 none

8) RD (pin no-32)

- Active low signal

- Read strobe indicates that processor is performed memory, I/O read cycle depending on s2
pin

9) Ready (pin no-22)

- When high-- ☐ carry out it normal operation
- When low - ☐ freezes its bus & enters a wait state

10) INTR (pin no-18) Interrupt request

- High level triggered interrupt request i/p
- Checked last clock to check availability of request
- If request is not occurred, processor enters interrupt acknowledge cycle **11) TEST (pin no-23)**

- This i/p is examined by 8086 wait instruction

- If TEST i/p ☐ low - ☐ execution continued -- ☐ if not then processor enter /wait in
idle state **12) NMI (pin no-17) Non mask able interrupt**

- NMI is not mask able internally by software

- A transition from low- ☐ high initiates the interrupt at the end of current instruction - It is edge triggered i/p interrupt causes type-2 interrupt

13) MN / MX

- Pin indicates the operating mode of 8086
- There are 2 modes of OS i.e maximum and minimum modes
- MN/MX - ☐ high 8086 operating in -- ☐ minimum mode
- MN/MX - ☐ low 8086 operating in -- ☐ maximum mode

ADDRESSING MODES OF 8086

Addressing mode specifies the way an operand (data) is accessed for an instruction in assembly language programming. It tells the CPU where the operand is located — in a register, memory, or given directly in the instruction.

The **8086** supports the following **addressing modes**:

1. Immediate Addressing Mode

- Operand is given directly in the instruction.
- Fastest method because data is already available.

Example:

```
MOV AL, 25H ; AL ← 25H  
MOV BX, 1234H ; BX ← 1234H
```

2. Register Addressing Mode

- Operand is stored in a register.
- The instruction specifies the register name.

Example:

```
MOV AX, BX ; AX ← BX  
ADD DL, CL ; DL ← DL + CL
```

3. Direct Memory Addressing Mode

- The offset address (within a segment) of the operand is given directly in the instruction.

Example:

```
MOV AL, [5000H] ; AL ← contents of memory at DS:5000H  
MOV [6000H], DL ; Store DL at DS:6000H
```

4. Register Indirect Addressing Mode

- Memory address of operand is held in a register (BX, SI, DI, BP).
- Physical address is computed using the register and DS or SS segment.

Example:

MOV AL, [BX] ; AL ← contents at DS:[BX]
MOV [SI], DL ; Store DL at DS:[SI]

5. Indexed Addressing Mode

• Offset address = Index Register (SI/DI) + Displacement.
MOV AL, [SI+200H] ; AL ← contents at DS:(SI+200H)
MOV DL, [DI+1050H]

6. Based Addressing Mode

• Offset address = Base Register (BX/BP) + Displacement. Example:

MOV AL, [BX+300H] ; AL ← DS:(BX+300H)
MOV DL, [BP+500H] ; DL ← SS:(BP+500H)

7. Based Indexed Addressing Mode

• Offset address = Base Register + Index Register.

Example:

MOV AL, [BX+SI] ; AL ← DS:(BX+SI)
MOV DL, [BP+DI] ; DL ← SS:(BP+DI)



Base Register Index Register Index



Base Address

8. Based Indexed with Displacement

EA Memory Datum

• Offset address = Base Register + Index Register + Displacement.

Example:

MOV AL, [BX+SI+200H]
MOV DL, [BP+DI+1000H]

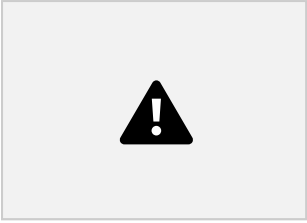
Base Register Index Register Displacement



Index



Base Address



EA Memory



Datum